

ARTIFICIAL INTELLIGENCE

Lecture Set 06
Local Search

Dr. Maria Siddiqua
Assistant Professor
Department of AI & DS
FAST-NUCES Karachi
maria.siddiqua@nu.edu.pk

Course Contents

1. Introduction to AI
2. Agents and Environments
3. Problem Solving by Searching
4. Uninformed Search
5. Informed Search
- **6. Local Search**
7. Constraint Satisfaction Problems
8. Adversarial Search
9. Logical Agents
10. First-order Logic
11. Uncertain Knowledge and Reasoning
12. Supervised, Unsupervised, and Reinforcement Learning
13. Recent Trends in AI and Applications of AI Algorithms

Table of Contents

- Hill Climbing Search
- Variants of Hill Climbing Search
- Simulated Annealing
- Local Beam Search
- Genetic Algorithm
- Real Life Examples of Local Search

Classical Search

The search problems we have considered during the *Uninformed/Informed Search* have the following properties:

- ▶ **Observable**

- If an agent's sensors give it access to the complete state of the environment at each point in time, then we say that the task environment is fully observable.

- ▶ **Deterministic**

- If the next state of the environment is completely determined by the current state and action executed by the agent, then we say the environment is deterministic.

- ▶ **Known**

- In a known environment, the outcomes for all actions are given and the agent won't have to learn how it works in order to make good decisions.

Classical search algorithms *explore the search space systematically* and the solution of the search problem is a sequence of *actions* representing paths from an initial state to the goal state.

Iterative Improvement Algorithms

In many optimization problems, **path** is irrelevant;
the goal state itself is the solution

Then state space = set of “complete” configurations;
find **optimal** configuration, e.g., TSP
or, find configuration satisfying constraints, e.g., timetable

In such cases, can use **iterative improvement** algorithms;
keep a single “current” state, try to improve it

Constant space, suitable for online as well as offline search

Local Search

- ▶ There exists some problems where the path to the goal is irrelevant.
- ▶ For example,

In the *8-queens* problem what matters is the final configuration of queens, not the order in which they are added.

- ▶ If the path to the goal does not matter, we might consider a different class of algorithms (Local Search), ones that do not worry about paths at all.
- ▶ Local search algorithms evaluate and modify one or more current states rather than systematically exploring paths from an initial state.
- ▶ **Local Search**
 - Keep track of single current state
 - Move only to neighboring states
 - Path followed by the search are not retained
- ▶ **Advantages**
 - Use very little memory
 - Can often find reasonable solutions in large or infinite state spaces for which systematic algorithms are unsuitable
 - Useful for solving pure optimization problems, in which the aim is to find the best state according to an objective function

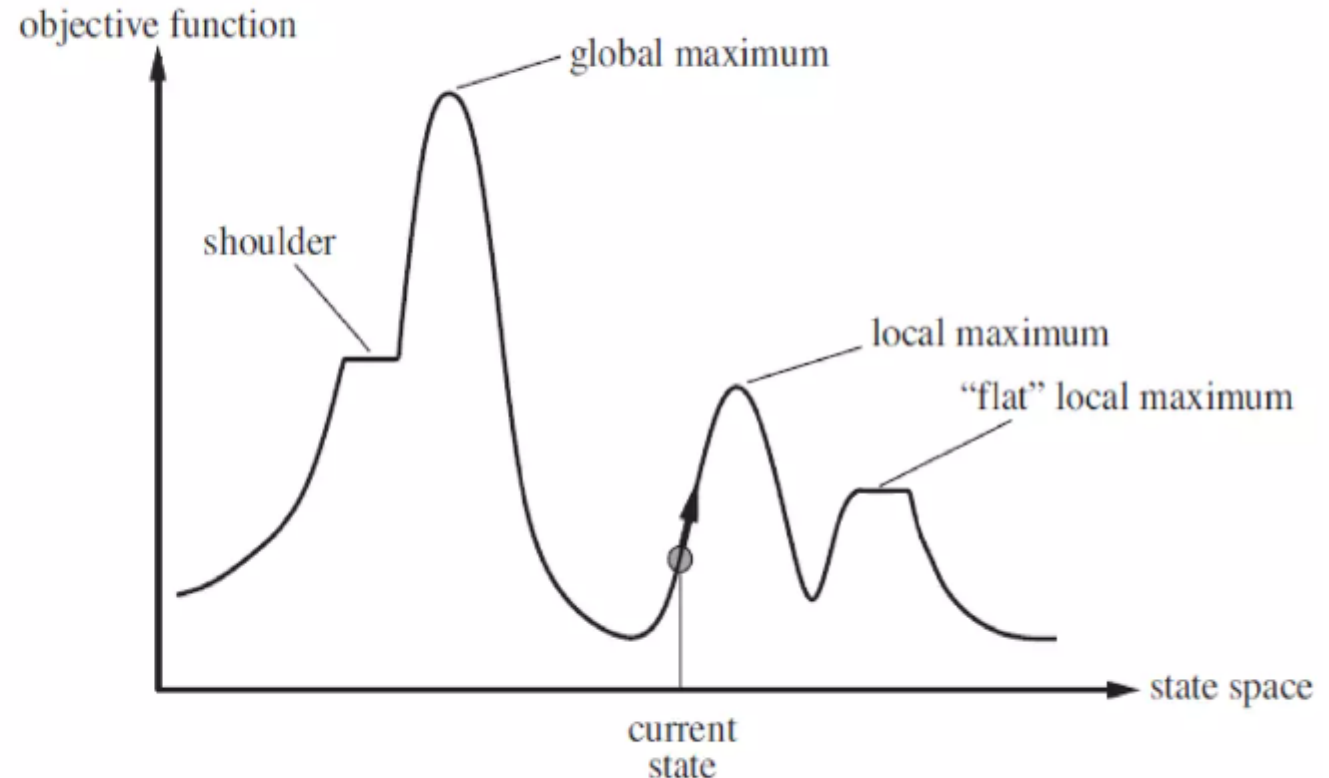
State-space Landscape

Components >>

- ▶ **Location** – state
- ▶ **Elevation** – heuristic cost function or, objective function.

An optimal algorithm always finds a global maximum(for objective function)/minimum(for heuristic cost function)

- ▶ **Local Maxima** – a peak that is higher than each of its neighboring states but lower than the global maximum
- ▶ **Plateaux** – a plateau is a flat area of the state-space landscape. It can be a *flat local maximum* from which no uphill exit exists, or a *shoulder* from which progress is possible.



Hill-climbing Search >> Steepest-ascent Version

function **Hill-Climbing**(*problem*) returns a state that is a local maximum

inputs: *problem*, a problem

local variables: *current*, a node

neighbor, a node

current ← Make-Node(Initial-State[*problem*])

loop do

neighbor ← a highest-valued successor of *current*

if Value[neighbor] ≤ Value[current] then return State[*current*]

current ← *neighbor*

end

- ▶ Hill-climbing search algorithm is simply a loop that continually *moves in the direction of increasing value* - that is *uphill*.
- ▶ It terminates when it reaches a peak where no neighbor has a higher value.
- ▶ It doesn't maintain a search tree, so the data structure for the current node need only record the state and the value of the objective function/heuristic cost function.
- ▶ Can randomly choose among the set of best successors, if multiple have the best value.
- ▶ It doesn't look ahead beyond the immediate neighbors of the current state.
- ▶ Also known as **Greedy Local Search**

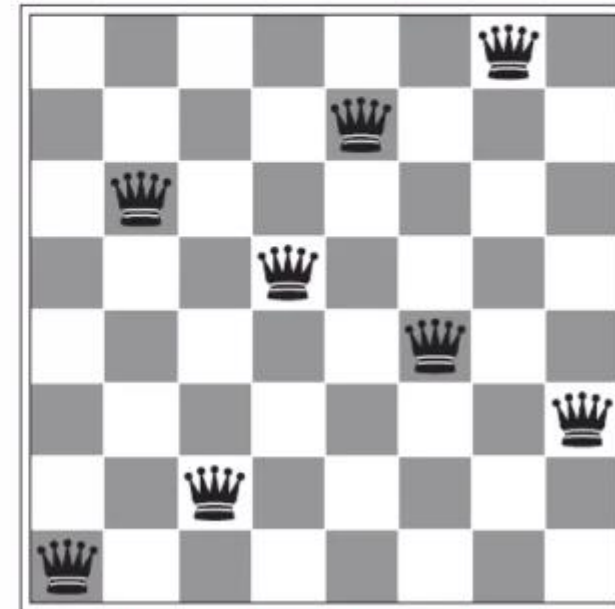
Hill-climbing Search >> Example

▶ 8-queens problem:

- Local search algorithms use a complete-state formulation, where each state has 8-queens on the board one per column.
- Successors of a state are all possible states generated by moving a single queen to another square in the same column. (8x7=56 successors)
- **Heuristic cost function h is the number of pairs of queens that are attacking each other, either directly or indirectly.**
- The global minima of this heuristic function i.e. zero occurs only at perfect solution.

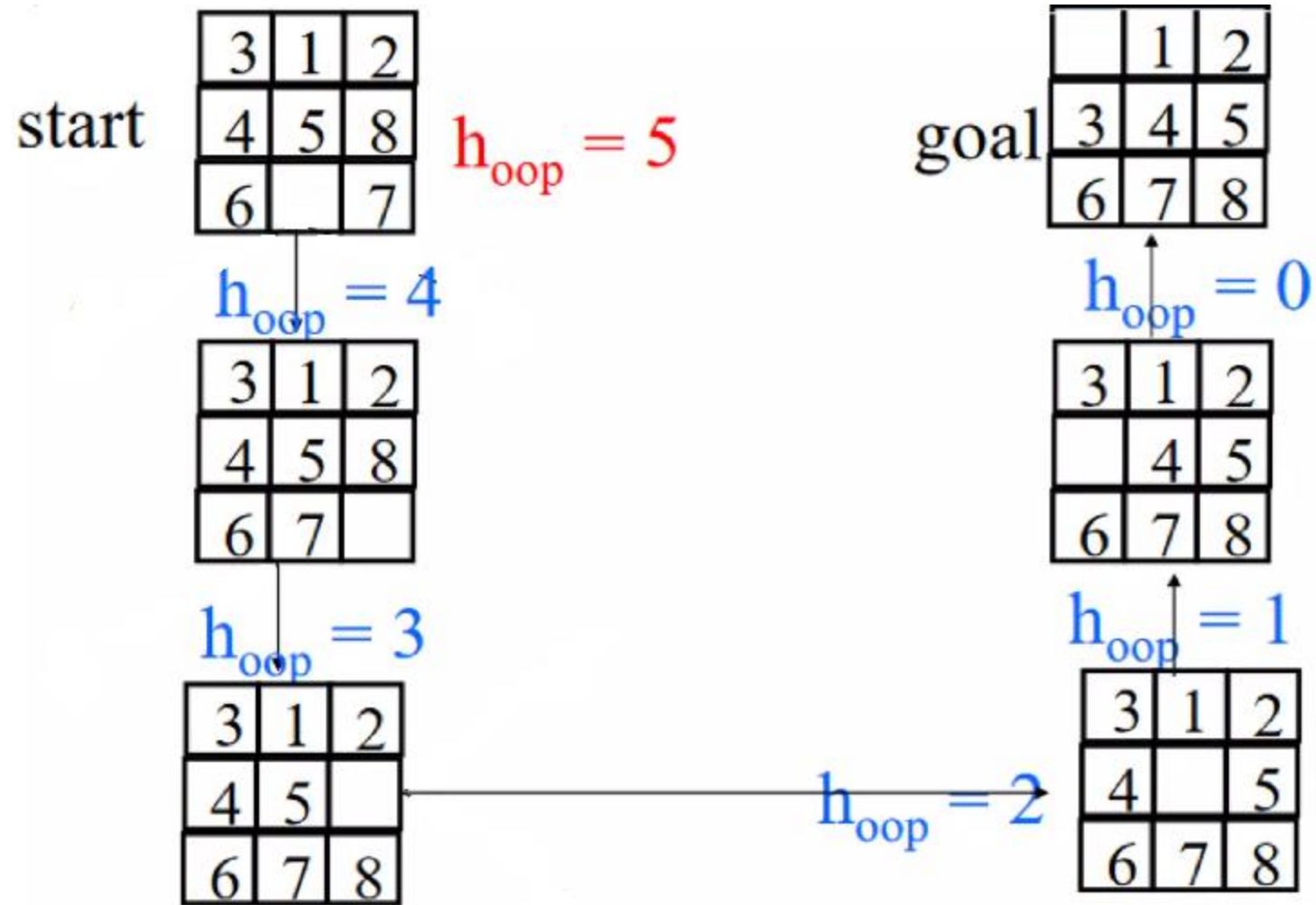
18	12	14	13	13	12	14	14
14	16	13	15	12	14	12	16
14	12	18	13	15	12	14	14
15	14	14	♙	13	16	13	16
♙	14	17	15	♙	14	16	16
17	♙	16	18	15	♙	15	♙
18	14	♙	15	15	14	♙	16
14	14	13	17	12	14	12	18

Initial State, $h=17$



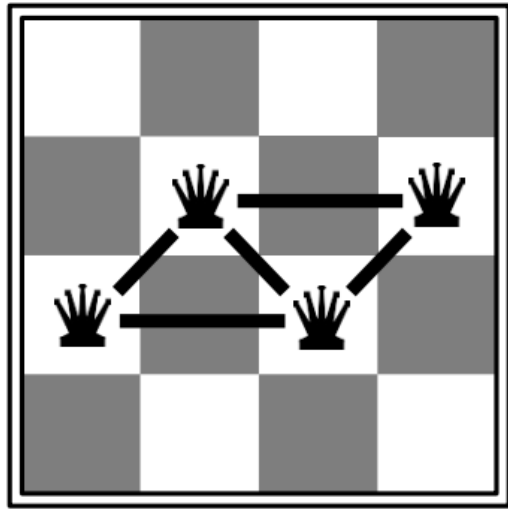
Local minima, $h=1$

Hill-climbing Search >> 8-Puzzle

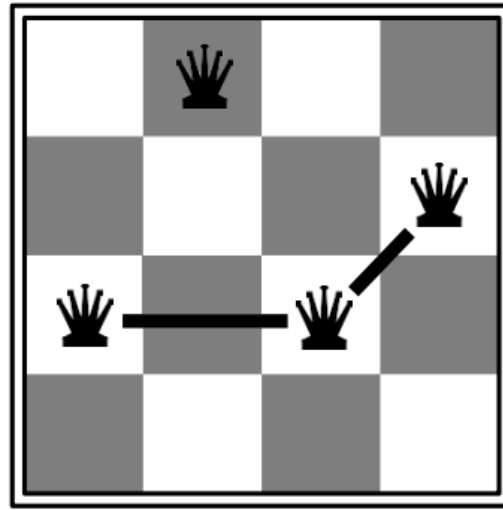
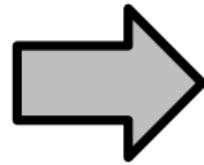


Hill-climbing Search >> 4-Queen

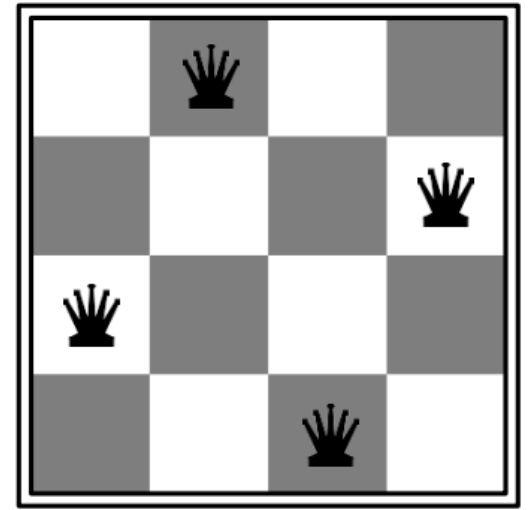
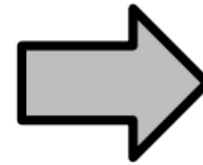
Move a queen to reduce number of conflicts



$h = 5$



$h = 2$



$h = 0$

Hill-climbing Search >> Drawbacks

Hill-climbing search often gets stuck for the following reasons:

▶ Local Maxima >>

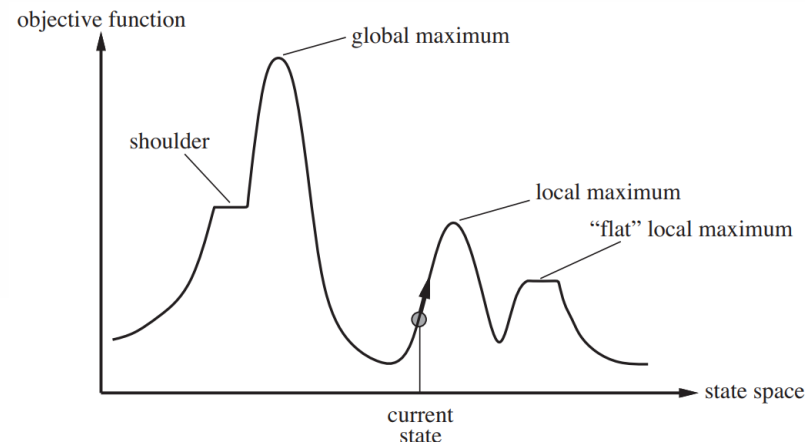
- ▶ It is a peak that is higher than each of its neighboring states but lower than the global maximum.
- ▶ For 8-queens problem at local minima, each move of a single queen makes the situation worse.

▶ Ridges >>

- ▶ Sequence of local maxima difficult for greedy algorithms to navigate

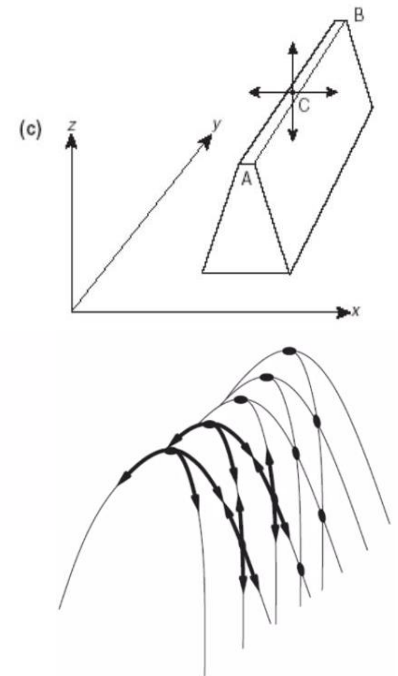
▶ Plateaux >>

- ▶ A plateau is a flat area of the state-space landscape. It can be a flat local maximum or a shoulder.



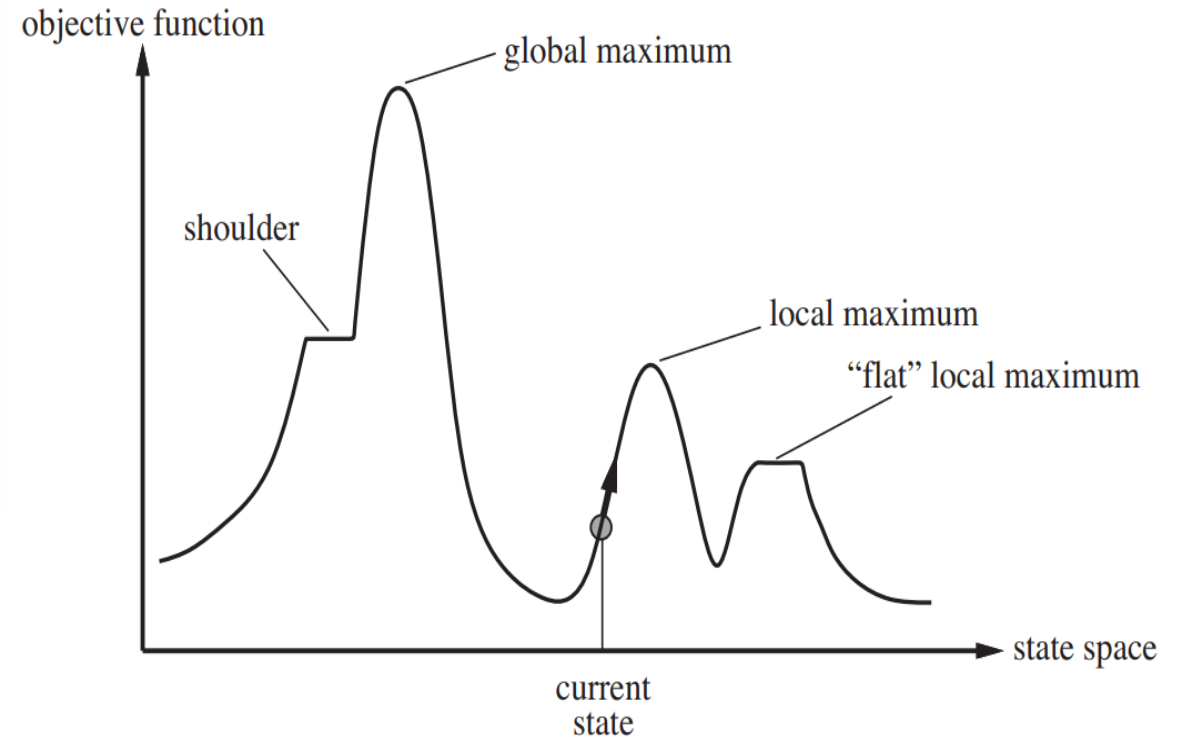
- Cause difficulties for hill-climbing methods
- B is higher than A.

- At C, the hill-climber can't find a higher point North, South, East or West, so it stops.



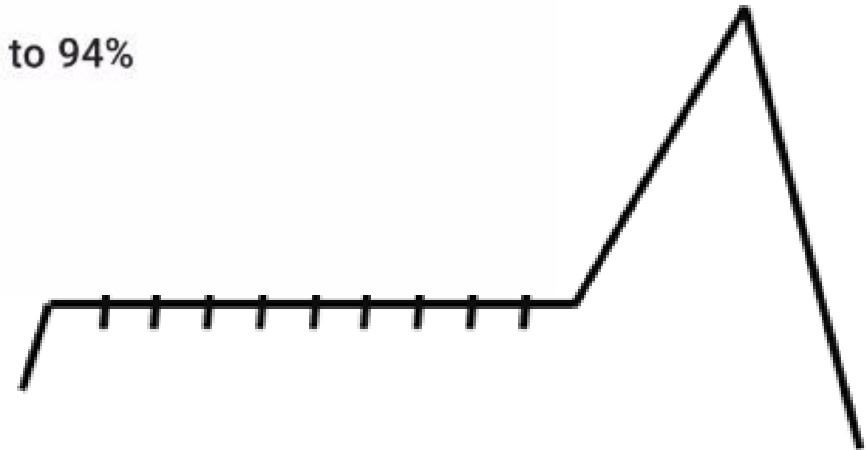
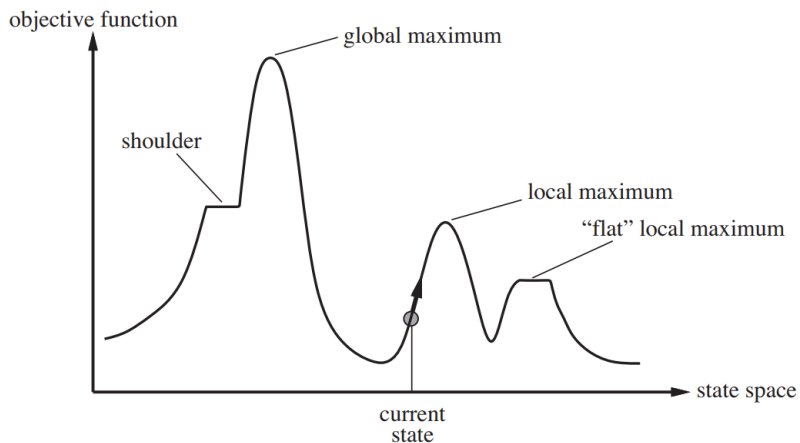
Hill-climbing Search >> Performance

- ▶ Randomly generated 8-queens starting state...
 - ▶ 14% the time it solves the problem
 - ▶ 86% of the time it gets stuck at a local minimum
- ▶ However...
- Takes only 4 steps on average when it succeeds
 - And 3 on average when it gets stuck
 - Not bad (for a state space with $8^8 \sim 17$ million states)



Hill-climbing Search >> Sideway Move Version

- ▶ If no downhill (uphill) moves, allow sideways moves in hope that algorithm can escape
 - Need to place a limit on the possible number of sideways moves to avoid infinite loops
- ▶ For 8-queens
 - Now allow sideways moves with a limit of 100
 - Raises percentage of problem instances solved from 14 to 94%
 - However....
 - 21 steps for every successful solution
 - 64 for each failure



Hill-climbing Search >> Variations

► Stochastic hill-climbing

- Random selection among the uphill/downhill moves.
- The selection probability can vary with the steepness of the uphill move.

Sample Algorithm [Uphill version] >>

Procedure stochastic hill-climber

begin

$t=0$

 select a current string v_c at random

 evaluate v_c

 repeat

 select the string v_n from the neighborhood of v_c

 select v_n with probability $\frac{1}{1 + e^{\frac{eval(v_c) - eval(v_n)}{T}}}$

$t=t+1$

 until $t=MAX$

end

$eval(v_c) = 107$, $T = \text{const} = 10$

$eval(v_n)$	$eval(v_c) - eval(v_n)$	p
80	27	0.06
100	7	0.33
107	0	0.50
120	-13	0.78
150	-43	0.99

► First-choice hill-climbing

- stochastic hill climbing by generating successors randomly until a better one is found
- Useful when there are a very large number of successors

► Random-restart hill-climbing

- If at first you don't succeed, try, try again.
- Tries to avoid getting stuck in local maxima.

Simulated Annealing

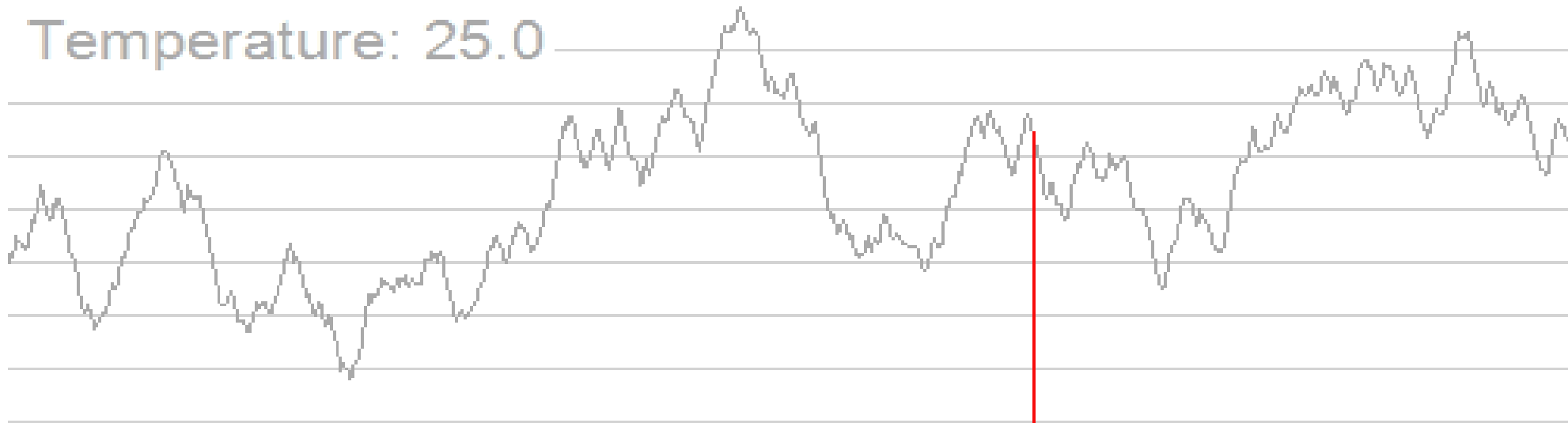
- ▶ A hill-climbing algorithm that never makes downhill moves (for maximization problem) toward states with lower value (or higher cost) is guaranteed to be incomplete, because it can get stuck on a local maximum.
- ▶ In contrast, a purely random walk—that is moving to successor chosen uniformly at random from the set of successors—is complete but extremely inefficient.
- ▶ Simulated Annealing is such an algorithm that combines both hill climbing with a random walk in such a way that yields both efficiency and completeness.
- ▶ In metallurgy, annealing is the process used to temper or harden metals and glass by heating them to a high temperature and then gradually cooling them, thus allowing the material to reach a low energy crystalline state.
- ▶ The simulated-annealing solution is to start by shaking hard (i.e. high temperature) and then gradually reduce the intensity of the shaking (i.e. lower the temperature).
- ▶ Instead of picking the best move, it picks a random move.
 - If the move improves the situation, it is always accepted.
 - Otherwise, the algorithm accepts the move with some probability less than 1.
- ▶ The probability decreases exponentially with the badness of the move – the amount ΔE by which the evaluation is worsened.
- ▶ The probability also decreases as the temperature T goes down. As a result bad moves more likely to be allowed at the start when T is high, and they become more unlikely as T decreases.
- ▶ If the schedule lowers T slowly enough, the algorithm will find a global optimum with probability approaching 1.

Simulated Annealing

- In optimization algorithms, annealing is used **metaphorically** to describe a process where the exploration of the search space **starts** with a **high level of randomness** and **gradually reduces** to a more focused search as the algorithm progresses.
- The key idea is to allow the algorithm to **explore both good and bad solutions** early in the search, and then **gradually shift towards exploiting** the better solutions as the search progresses.
- In simulated annealing specifically, the temperature parameter plays a crucial role in controlling the annealing process.
- At **high temperatures**, the algorithm is more likely to **accept worse solutions**, allowing for exploration of the search space and escape from local optima.
- As the **temperature decreases** over time according to a predefined schedule (annealing schedule), the algorithm becomes less likely to accept worse solutions and focuses more on **exploiting** the **better solutions** found so far.

Simulated Annealing

- Explore successors wildly randomly: High temp
- As time goes by , explore less wildly: Cool Down
- Until there's a time when things settle: Cold



Simulated Annealing >> Algorithm

function SIMULATED-ANNEALING(*problem*, *schedule*) **returns** a solution state

inputs: *problem*, a problem

schedule, a mapping from time to “temperature”

current $\leftarrow$ MAKE-NODE(*problem*.INITIAL-STATE)

for $t = 1$ **to** ∞ **do**

$T \leftarrow$ *schedule*(t)

if $T = 0$ **then return** *current*

next $\leftarrow$ a randomly selected successor of *current*

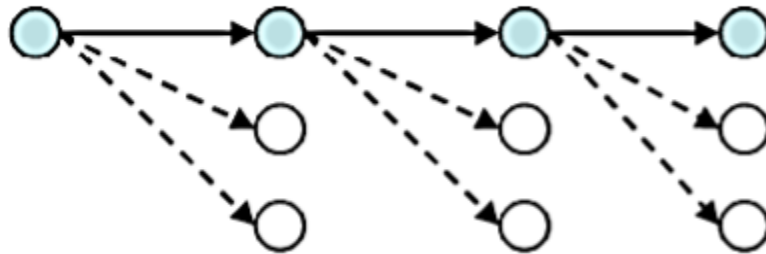
$\Delta E \leftarrow$ *next*.VALUE $-$ *current*.VALUE

if $\Delta E > 0$ **then** *current* $\leftarrow$ *next*

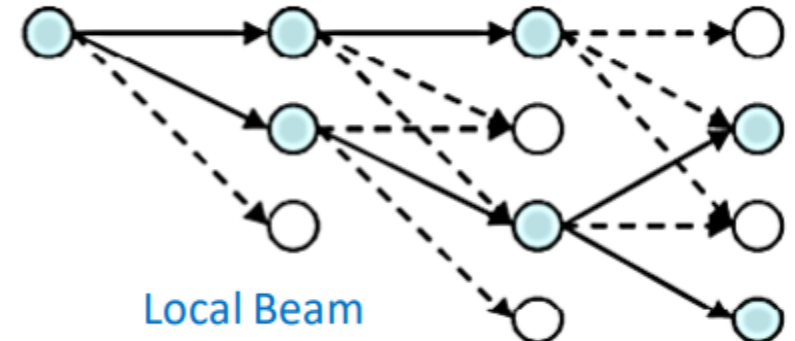
else *current* $\leftarrow$ *next* only with probability $e^{\Delta E/T}$

Local Beam Search

- ▶ Because of memory limitations, we are just maintaining one node in memory in our previous local search algorithms.
- ▶ The local beam search keeps track of k states rather than just one.
 - ▶ It begins with k randomly generated states.
 - ▶ At each step, all the successors of all k states are generated. If any one is a goal, the algorithm halts. Otherwise it selects the k best successors from the complete list and repeats.
- ▶ In a random restart search each search process runs independently of the others; while in local beam search useful information is passed among the parallel search threads.
- ▶ The algorithm quickly abandons unfruitful searches and moves its resources to where the most progress is being made.
- ▶ **Problem:**
 - It can suffer from a lack of diversity among the k states - they can quickly become concentrated in a small region of the state space, making the search little more than an expensive version of hill-climbing.
- ▶ **Solution:**
 - **Stochastic Beam Search** : Instead of choosing the best k from the pool of candidate successors, stochastic beam search chooses k successors at random, with the probability of choosing a given successor being an increasing function of its value.



Greedy Local



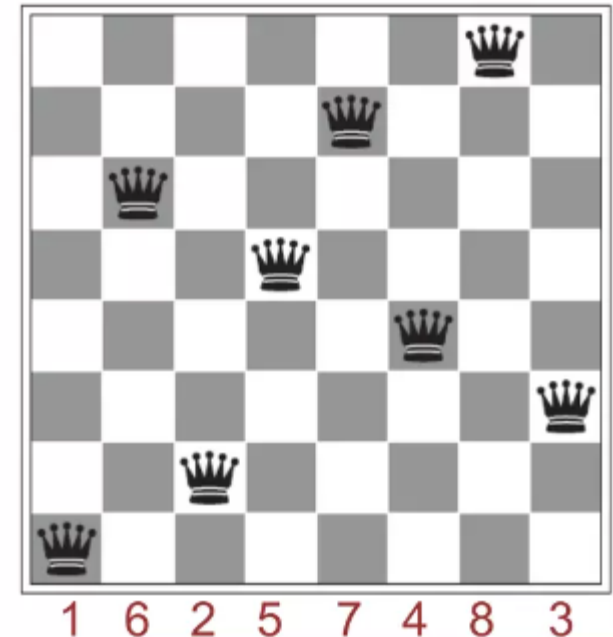
Local Beam

Genetic Algorithm (GA)

- ▶ A *Genetic Algorithm* is a variant of stochastic beam search in which successor states are generated by combining two parents rather than by modifying a single state.
- ▶ **Population** >> GAs begin with a set of k randomly generated states, called population.
- ▶ **Individual** >> Each state or, individual is represented as a string over finite alphabet.

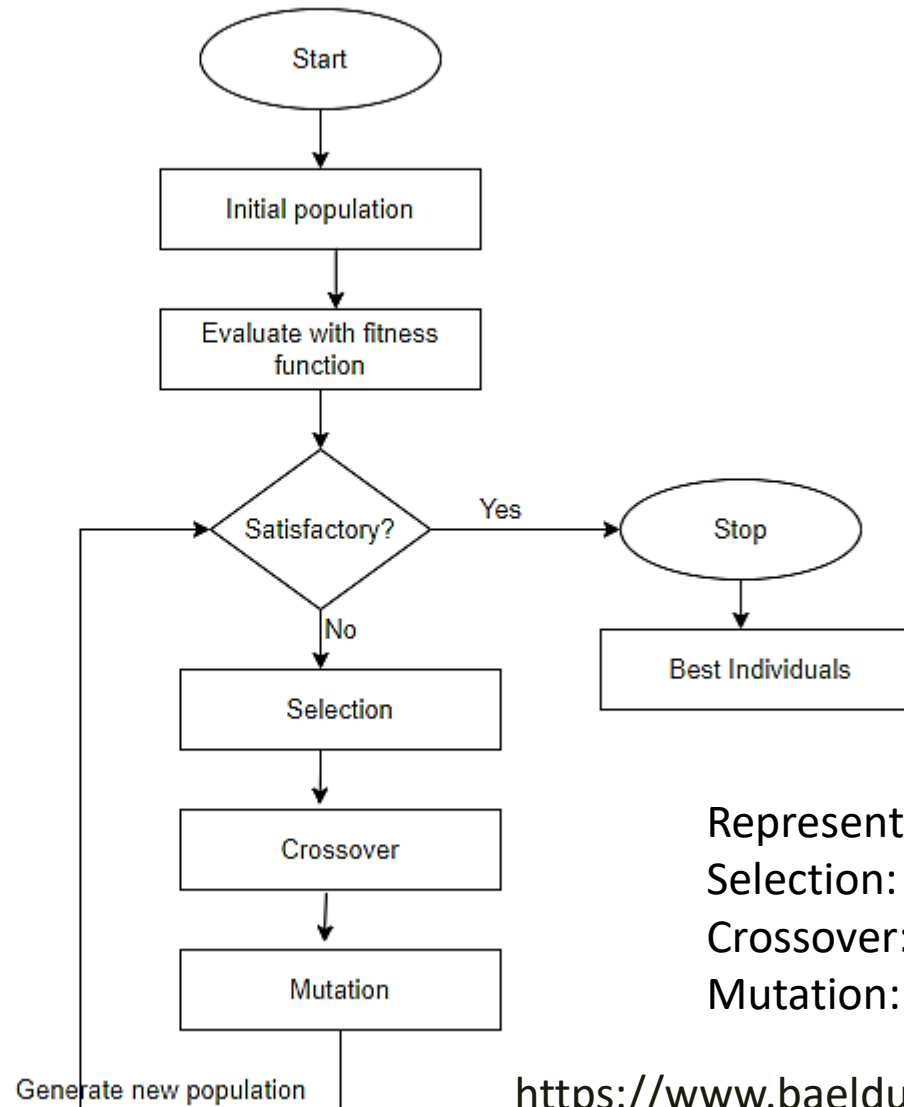
For example, 8-queens state must specify the positions of 8-queens, each in a column of 8 squares.

- ▶ Genetic algorithms combine an uphill tendency with random exploration and exchange of information among parallel search threads.



Genetic Algorithm (GA)

Genetic algorithms follow the natural process of evolution, and are described as follows:



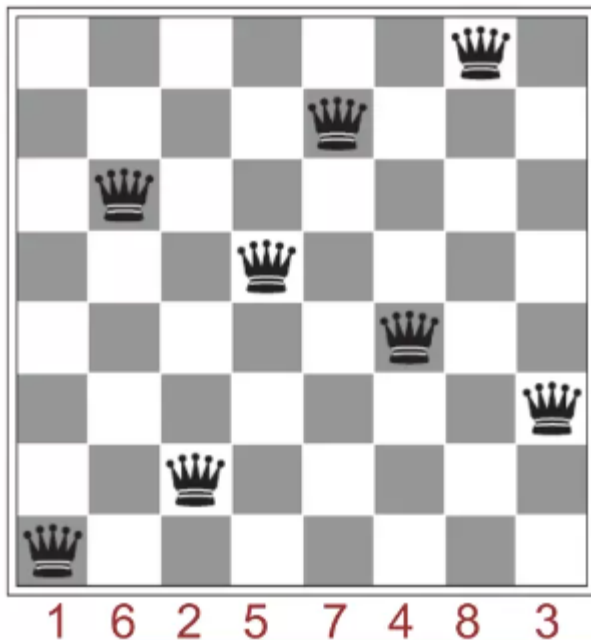
Representation/Encoding: binary, real value
Selection: Roulette Wheel (fitness/total fitness)
Crossover: one point, two point, multipoint, uniform
Mutation: bitwise mutation

Genetic Algorithm (GA) >> Fitness Function

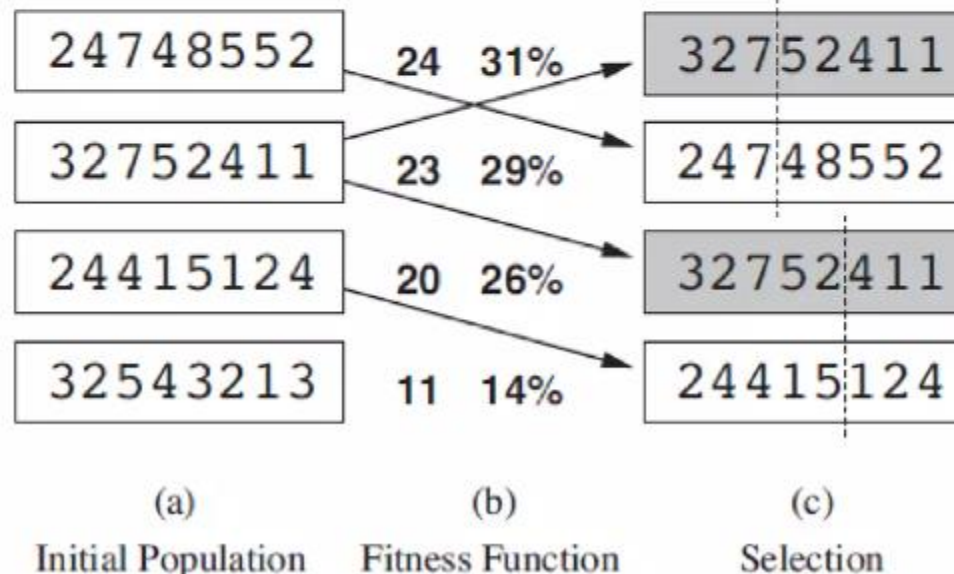
- Each state/individual is rated by the objective/fitness function. A fitness function should return higher values for better states.

For 8-queens, fitness function = number of not-attacking pairs of queens, which has a value of 28 for solution.

- In this variant, we assume that the probability of being chosen for reproducing is directly proportional to the fitness score.



Survival of the fittest



Represent states

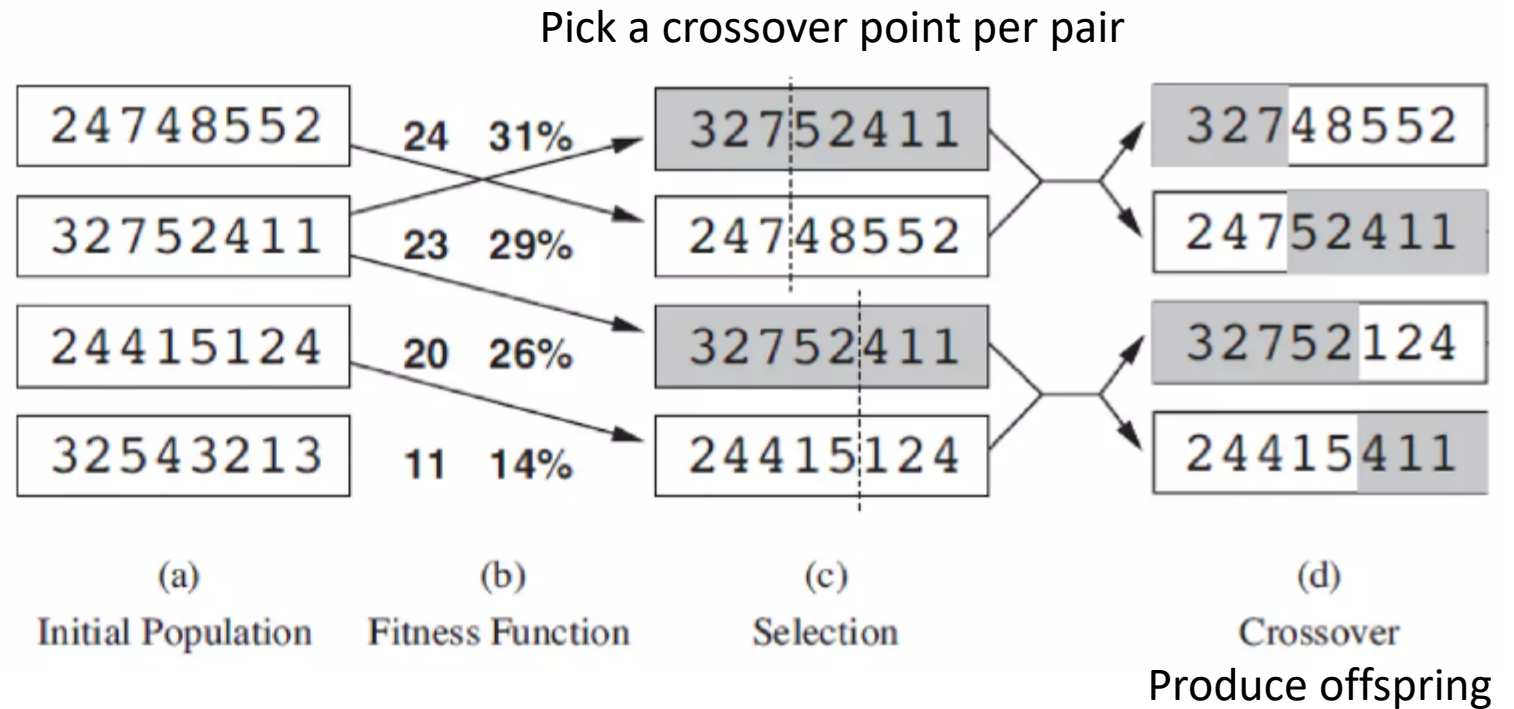
Compute fitness function

Sum of fitness=24+23+20+11= 78

- Compute probability of being chosen from fitness function ($f/\text{sum of } f$)
- $24/78 = 0.307$
- Normalize:
- $0.307 \times 100 = 30.7\%$ chance of being chosen probably
- Randomly choose two pairs to reproduce based on probabilities.

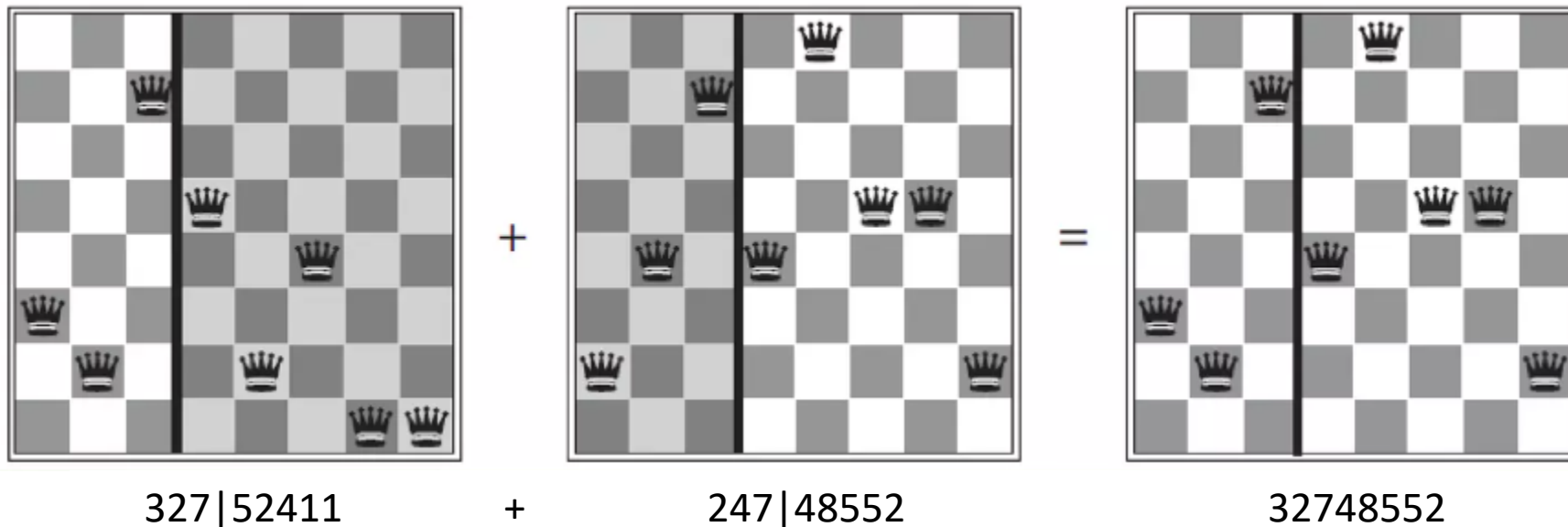
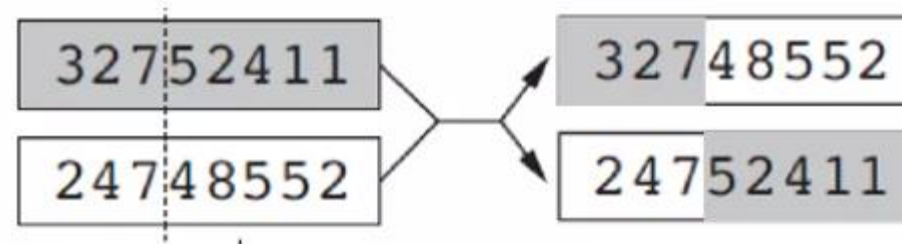
Genetic Algorithm (GA) >> Reproduction & Crossover

- ▶ Pairs are selected at random for reproduction, in accordance with the calculated probabilities of the previous step.
- ▶ For each pair to be mated, a crossover point is chosen randomly from the positions in the string.
 - In our case, the crossover points are after the third digit in the first pair and after the fifth digit in the second pair.
 - In our variant, each mating of two parents produces two offsprings.



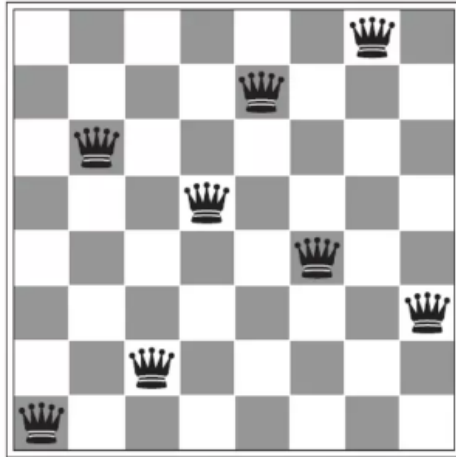
Genetic Algorithm (GA) >> Reproduction & Crossover

- ▶ The population is quite diverse early on in the process, so crossover frequently takes large steps in the state space early in the search process and smaller steps later on when most individuals are quite similar.
- ▶ The following figure illustrates that when two parent states are quite different, the crossover operation can produce a state that is a long way from either parent state.



Genetic Algorithm (GA) >> Mutation

- ▶ Each location is subject to random mutation with a small independent probability.
- ▶ For 8-queens problem, we need to choose a queen at random and moving it to a random square in its column.
- ▶ In our example, one digit was mutated in the first, third and fourth offspring.



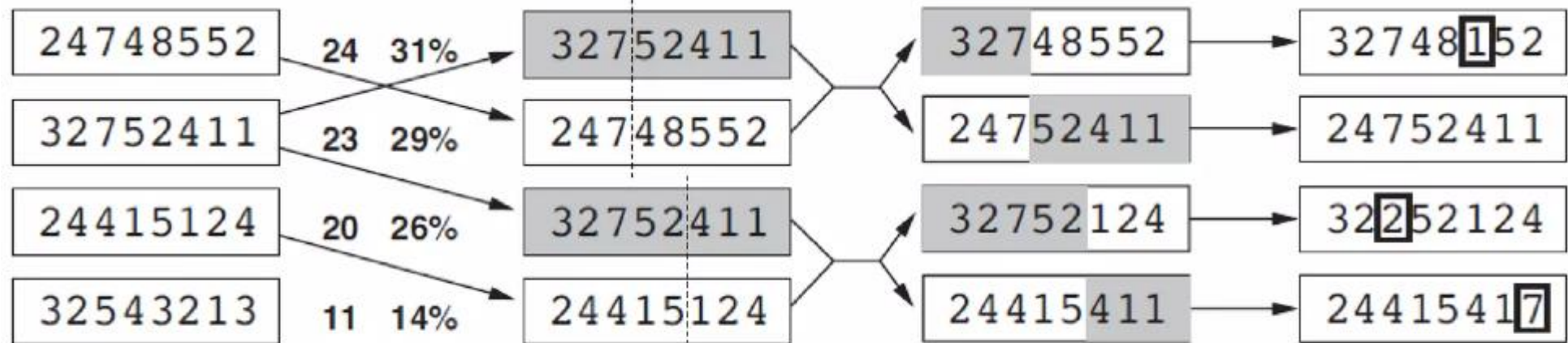
1 6 2 5 7 4 8 3

Representation/Encoding: binary, real value

Selection: Roulette Wheel (fitness/total fitness)

Crossover: one point, two point, multipoint, uniform

Mutation: bitwise mutation



(a)

Initial Population

(b)

Fitness Function

(c)

Selection

(d)

Crossover

(e)

Mutation

Genetic Algorithm (GA) >> Algorithm

function GENETIC-ALGORITHM(*population*, FITNESS-FN) **returns** an individual

inputs: *population*, a set of individuals

FITNESS-FN, a function that measures the fitness of an individual

repeat

new_population $\leftarrow$ empty set

for $i = 1$ **to** SIZE(*population*) **do**

$x \leftarrow$ RANDOM-SELECTION(*population*, FITNESS-FN)

$y \leftarrow$ RANDOM-SELECTION(*population*, FITNESS-FN)

child $\leftarrow$ REPRODUCE(x, y)

if (small random probability) **then** *child* $\leftarrow$ MUTATE(*child*)

add *child* to *new_population*

population $\leftarrow$ *new_population*

until some individual is fit enough, or enough time has elapsed

return the best individual in *population*, according to FITNESS-FN

function REPRODUCE(x, y) **returns** an individual

inputs: x, y , parent individuals

$n \leftarrow$ LENGTH(x); $c \leftarrow$ random number from 1 to n

return APPEND(SUBSTRING($x, 1, c$), SUBSTRING($y, c + 1, n$))

Genetic Algorithm (GA) >> Crossover vs Mutation

Aspect	Crossover	Mutation
Purpose	Combine genetic material from parents	Introduce random changes
Mechanism	Exchange genetic material between parents	Alter genetic material randomly
Role	Promotes exploration	Introduces diversity

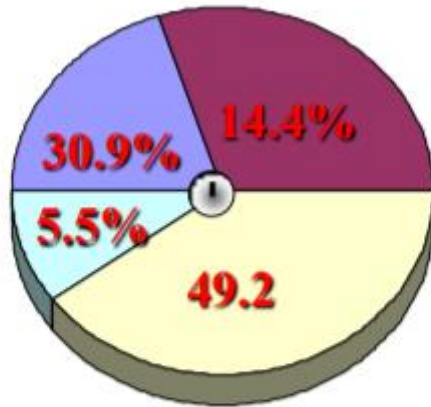
- Crossover involves the recombination of genetic material from parent individuals to create offspring with traits inherited from both parents.
- Mutation introduces random changes to individual chromosomes to explore new solutions and maintain genetic diversity.
- Both operators play complementary roles in genetic algorithms, contributing to the effectiveness of the search process and the discovery of optimal or near-optimal solutions.

Genetic Algorithm (GA) >> Function Maximization

Maximize the function $f(x)=x^2$, where x value range from 0-31

GA approach:

- Representation: binary code, e.g. 01101 $\leftrightarrow$ 13
- Population size: 4
- 1-point xover, bitwise mutation
- Roulette wheel selection
- Random initialisation



Weighted Roulette wheel

defines the roulette wheel
 $F(x)/\text{Total } F(x)$

$F(x)/\text{Average } F(x)$

Number of times the parent will get
selected if wheel is spun 'n' times
 $AC = \text{round}(EC)$

String No	Initial Population	X Value	F(x) value	Probability count	Expected count	Actual Count
1	01101	13	169	0.14	0.58	1
2	11000	24	576	0.49	1.97	2
3	01000	8	64	0.06	0.22	0
4	10011	19	361	0.31	1.23	1
Total			1170	1	4	
Average			293			

We see that if the Roulette wheel is spun four times, we'll get 24 twice and 13 and 19 once. So possible parental combinations are (24,13) and (24,19).

Genetic Algorithm (GA) >> Function Maximization

Maximize the function $f(x)=x^2$, where x value range from 0-31

(24,13) and (24,19)

String 2	1 1 0 0 0	Parental combination 1
String 1	0 1 1 0 1	
String 2	1 1 0 0 0	Parental combination 2
String 4	1 0 0 1 1	

Parents

Offsprings

String 2	1 1 0 0 0	1 1 0 0 1
String 1	0 1 1 0 1	0 1 1 0 0
String 2	1 1 0 0 0	1 1 0 1 1
String 4	1 0 0 1 1	1 0 0 0 0

String No	Offspring 1	X Value	F(x) value
1	0 1 1 0 0	12	144
2	1 1 0 0 1	25	625
3	1 1 0 1 1	27	729
4	1 0 0 0 0	10	256

We can see that the maximum $f(x)$ value has increased from 576 to 729.

Genetic Algorithm (GA) >> Knapsack Problem

- Let's say, you are going to spend a month in the wilderness. Only thing you are carrying is the backpack which can hold a maximum weight of **30 kg**. Now you have different survival items, each having its own “Survival Points” (which are given for each item in the table). So, your objective is maximize the survival points.
- Here is the table giving details about each item.

ITEM	WEIGHT	SURVIVAL POINTS
SLEEPING BAG	15	15
ROPE	3	7
POCKET KNIFE	2	10
TORCH	5	5
BOTTLE	9	8
GLUCOSE	20	17

- Choose an initial population of four
- Then choose the best three subsequently (Encoding hint: 1 if the item is chosen, 0 otherwise)
- Decide crossover point
- Chance of mutation = 50 items in 100 generations. (bitwise mutation).
- Perform three iterations

Genetic Algorithm (GA) >> Knapsack Problem

Knapsack Problem Date _____ 20____

* Backpack can hold a maximum weight of 30 kg.
 * Objective is to maximize the survival points assigned to each survival item that can be carried in backpack.

Item	Weight	Survival points
Sleeping bag	15	15
Rope	3	7
Pocket Knife	2	10
Torch	5	5
Bottle	9	8
Glucose	20	17

First Iteration

① Initial population: Taking four binary strings with weight ≤ 30 . There are six items so six bits are required.

100110
 000011
 011101
 111100

② Fitness Evaluation: calculate fitness using survival points

100110 $\rightarrow 15 + 0 + 0 + 5 + 8 + 0 = 28$
 000011 $\rightarrow 0 + 0 + 0 + 0 + 8 + 17 = 25$
 011101 $\rightarrow 0 + 7 + 10 + 5 + 0 + 17 = 39$
 111100 $\rightarrow 15 + 7 + 10 + 5 + 0 + 0 = 37$

③ Selection: Three strings with high fitness value i.e. string 3, 4, and 1 are selected.

Combination: string 1 and 3 and string 3 and 4

100110 011101
 011101 111100

④ Crossover: using one point crossover after three digits

100110 $\rightarrow$ 100101 011101 $\rightarrow$ 011100
 011101 $\rightarrow$ 011110 111100 $\rightarrow$ 111101

Generation 1: 100101, 011110, 011100, 111101

⑤ Mutation: In problem, chance of mutation = 50 items in 100 generations. If 50 items can be mutated in 100 generations, then 1 item will mutate in 2 generations. In problem, we are asked to do three iterations so we will get three generations. On average, there will be 0.5 mutations per generation.

Number of mutations = Number of generations $\times$ Average mutations per generation
 $= 3 \times 0.5 = 1.5$

In three iterations, approximately 1.5 mutations will be done. Since you can't perform a fraction of a mutation, you would typically round this to 1. Therefore only one bit mutation can be done in this entire iterative phase.

If you want you can mutate one bit in this iteration else do it in next two iterations. I have mutated in this iteration, therefore output of first iteration is:

100101 can mutate any one bit so flipping the
 011110 $\rightarrow$ first string's last bit from 1 to 0 $\rightarrow$ 011100
 011100
 111101

100100
011110
011100
111101

New this output of first iteration will be the input for second iteration.

* Second Iteration :-

① Initial Population: It will be the output of previous (first) iteration.

② Fitness Evaluation:-

100100 $\rightarrow 15 + 0 + 0 + 5 + 0 + 0 = 20$
 011110 $\rightarrow 0 + 7 + 10 + 5 + 8 + 0 = 30$
 011100 $\rightarrow 0 + 7 + 10 + 5 + 0 + 0 = 22$
 111101 $\rightarrow 15 + 7 + 10 + 5 + 0 + 17 = 54$

Genetic Algorithm (GA) >> Knapsack Problem

Date _____ 20 _____

③ Selection: String 2, 3, and 4 → Combination: String 2 and 3, 3 and 4

④ Crossover: using one point crossover after three digits

$$\begin{array}{ccc} 01110 & \nearrow & 01100 \\ 01100 & \searrow & 01110 \end{array} \quad \begin{array}{ccc} 01100 & \nearrow & 01101 \\ 11101 & \searrow & 11100 \end{array}$$

Generation 2: 01100, 01110, 01101, 11100

⑤ Mutation: No mutation as only one bit mutation was allowed which is already done.
Output of crossover will be given as input to third iteration.

* Third Iteration:

① Initial Population: output of second iteration

② Fitness Evaluation:-

$01100 \rightarrow 0+7+10+5+0+0 = 22$
 $01110 \rightarrow 0+7+10+5+8+0 = 30$
 $01101 \rightarrow 0+7+10+5+0+17 = 39$
 $11100 \rightarrow 15+7+10+5+0+0 = 37$

③ Selection: String 2, 3, and 4 → Combination: String 2 and 3, 3 and 4

④ Crossover: Using one point crossover after three digits

$$\begin{array}{ccc} 01110 & \nearrow & 01101 \\ 01101 & \searrow & 01110 \end{array} \quad \begin{array}{ccc} 01101 & \nearrow & 01100 \\ 11100 & \searrow & 11101 \end{array}$$

Generation 3: 01101, 01110, 01100, 11101

⑤ Mutation: N/A.

Final strings after applying Genetic Algorithm:

01101	30	39	→ Optimal solution as weight is within the limit of 30 kg and its corresponding survival points are also high. → Optimal solution was present in 2nd generation as well.
01110	19	30	
01100	10	22	
11101	45	54	

Weight survival points

Continuing the algorithm for the specified number of iterations provides valuable opportunities for selection, exploration, and refinement of solutions.

M.M.

Genetic Algorithm (GA) >> Properties

- ▶ Work well for mixed (continuous and discrete) problems
- ▶ They are less susceptible to get stuck at local optima
- ▶ Computationally expensive
- ▶ However, easy to perform in parallel
- ▶ No math in the process. The objective (fitness) function may be hard

Real Life Examples of Local Search

1. Hill Climbing Search:

- **Network Routing:** Hill climbing algorithms are used in network routing protocols to optimize the flow of data packets through computer networks. The algorithm adjusts routing paths based on real-time network conditions to minimize latency and congestion.
- **Supply Chain Optimization:** Hill climbing algorithms can be applied to optimize supply chain logistics, such as warehouse management and transportation routing. The algorithm iteratively adjusts inventory levels and delivery schedules to minimize costs and maximize efficiency.

2. Variants of Hill Climbing Search:

- **First-Choice Hill Climbing:** This variant is commonly used in machine learning and data mining applications for feature selection and parameter optimization tasks.
- **Stochastic Hill Climbing:** Stochastic hill climbing algorithms are used in optimization problems where randomness can help escape local optima, such as in scheduling and resource allocation problems.

3. Simulated Annealing:

- **Manufacturing Process Optimization:** Simulated annealing algorithms are used in manufacturing industries to optimize production processes, such as metal casting and heat treatment. The algorithm adjusts process parameters to minimize defects and improve product quality.
- **VLSI Circuit Design:** Simulated annealing algorithms are employed in the design of Very Large-Scale Integration (VLSI) circuits to optimize layout and minimize power consumption while meeting performance constraints.

4. Local Beam Search:

- **Natural Language Processing:** Local beam search algorithms are used in natural language processing tasks, such as part-of-speech tagging and named entity recognition. The algorithm explores multiple hypotheses simultaneously and selects the most likely ones based on contextual information.
- **Parallel Computing:** Local beam search algorithms can be applied in parallel computing environments to explore multiple solutions concurrently and identify the best-performing ones.

5. Genetic Algorithm:

- **Evolutionary Biology:** Genetic algorithms are used in evolutionary biology to model natural selection and genetic variation. They are used to study the evolution of species, population dynamics, and the spread of genetic traits.
- **Financial Trading:** Genetic algorithms are applied in financial trading to optimize trading strategies and portfolio management. The algorithm evolves trading rules and parameters to maximize returns while minimizing risks.

Reference Material

1. <https://medium.com/geekculture/genetic-algorithm-a-simple-example-161d788f0465>
2. <https://www.youtube.com/watch?v=96-u9s6D16k>
3. <https://www.youtube.com/watch?v=udN28wPqaZI>
4. https://www.youtube.com/watch?v=lnVJWW_NzFY

Questions

1. What is the basic principle behind Hill Climbing Search?
2. Explain how Hill Climbing Search explores the search space to find the optimal solution.
3. What are the advantages and disadvantages of Hill Climbing Search compared to other search algorithms?
4. Describe a scenario where Hill Climbing Search may get stuck in a local maximum or minimum.
5. Compare and contrast first-choice hill climbing and basic hill climbing algorithms.
6. How does stochastic hill climbing differ from traditional hill climbing algorithms?
7. Explain the concept of sideways moves in hill climbing algorithms and how they contribute to exploration.
8. Discuss the benefits of using adaptive hill climbing techniques in optimization problems.
9. What is simulated annealing and how does it differ from traditional hill climbing algorithms?
10. Explain the concept of temperature in simulated annealing and its role in exploration and exploitation.
11. Discuss the similarities and differences between simulated annealing and stochastic hill climbing algorithms.
12. Describe a real-world optimization problem where simulated annealing could be applied effectively.
13. What is local beam search and how does it differ from traditional hill climbing algorithms?
14. Explain the concept of beam width in local beam search and its impact on the search process.
15. Discuss the advantages and limitations of local beam search compared to other search algorithms.
16. Describe a scenario where local beam search would be a suitable approach for solving a problem.
17. What are the key components of a genetic algorithm and how do they mimic biological evolution?
18. Explain the concepts of selection, crossover, and mutation in the context of genetic algorithms.
19. Discuss the advantages of genetic algorithms in exploring complex search spaces and finding near-optimal solutions.
20. Describe a problem domain where genetic algorithms have been successfully applied and discuss the reasons for their effectiveness.

Programming Tasks

1. Implement a basic hill climbing algorithm to find the maximum or minimum of a given mathematical function.
2. Develop a Python program to visualize the search process of hill climbing algorithm using a 2D or 3D plot.
3. Implement first-choice hill climbing algorithm to solve the N-Queens problem, where the objective is to place N queens on an NxN chessboard without any queen attacking each other.
4. Develop a stochastic hill climbing algorithm to solve the Traveling Salesman Problem (TSP), where the objective is to find the shortest possible route that visits each city exactly once and returns to the origin city.
5. Implement simulated annealing algorithm to optimize the parameters of a mathematical function or a machine learning model.
6. Develop a Python program to visualize the cooling schedule and exploration-exploitation trade-off in simulated annealing algorithm.
7. Implement local beam search algorithm to solve the 8-Puzzle problem, where the objective is to rearrange the tiles of a 3x3 grid to match a given target configuration.
8. Develop a Python program to compare the performance of local beam search with basic hill climbing algorithm on a benchmark optimization problem.
9. Implement a genetic algorithm to solve the Knapsack problem, where the objective is to maximize the total value of items selected for a given weight constraint.
10. Develop a Python program to visualize the evolution of solutions over generations in a genetic algorithm using matplotlib or other plotting libraries.